FILE: C:\HCL_TRAINING\server\src\services\matchingEngine\scoringEngine.js

LINES: 216 | SIZE: 7.6 KB | LANGUAGE: JS

MODULE: 4-Factor Weighted Scoring Model with Verbatim Citation Extraction

```
const { aiProvider } = require('./aiProvider');

/**
 * Normalizes degree level for comparison
 */
function getDegreeRank(degreeStr) {
  if (!degreeStr) return 1;
  const lower = degreeStr.toLowerCase();
  if (lower.includes('phd') || lower.includes('doctor')) return 4;
  if (lower.includes('master') || lower.includes('m.s') || lower.includes('m.tech') || l
> ower.includes('mba')) return 3;
  if (lower.includes('bachelor') || lower.includes('b.s') || lower.includes('b.tech') ||
>  lower.includes('b.e')) return 2;
  return 1;
}

/**
 * Calculates Explainable Match Result
 */
async function calculateCandidateMatch(candidate, job, resumeText, customWeights = null)
>  {
  // 1. Resolve weights
  const weights = customWeights || {
    skills: 40,
    semantic: 30,
    experience: 20,
    qualification: 10,
  };

  const totalWeight = weights.skills + weights.semantic + weights.experience + weights.q
> ualification;
  const normSkillsWeight = weights.skills / totalWeight;
  const normSemanticWeight = weights.semantic / totalWeight;
  const normExpWeight = weights.experience / totalWeight;
  const normQualWeight = weights.qualification / totalWeight;

  // 2. Skill Alignment (Required & Preferred)
  const candidateSkills = (candidate.skills || []).map((s) => s.name.toLowerCase());
  const candidateSkillsMap = {};
  (candidate.skills || []).forEach((s) => {
    candidateSkillsMap[s.name.toLowerCase()] = s;
  });

  const matchedRequiredSkills = [];
  const missingRequiredSkills = [];
  const matchedPreferredSkills = [];
  const missingPreferredSkills = [];

  const mandatoryList = job.mandatorySkills || [];
  const preferredList = job.preferredSkills || [];

  // Check mandatory skills
  for (const mSkill of mandatoryList) {
    const sNameLower = mSkill.name.toLowerCase();
    const foundSkill = candidateSkillsMap[sNameLower];

    if (foundSkill) {
      matchedRequiredSkills.push({
        skill: mSkill.name,
        evidence: foundSkill.evidence || `Demonstrated competency in ${mSkill.name}`,
        confidence: foundSkill.confidence || 0.95,
        category: mSkill.category || 'Core Requirement',
      });
    } else {
      missingRequiredSkills.push(mSkill.name);
    }
```

```
63 |     }
64 |
65 |     // Check preferred skills
66 |     for (const pSkill of preferredList) {
67 |       const sNameLower = pSkill.name.toLowerCase();
68 |       const foundSkill = candidateSkillsMap[sNameLower];
69 |
70 |       if (foundSkill) {
71 |         matchedPreferredSkills.push({
72 |           skill: pSkill.name,
73 |           evidence: foundSkill.evidence || `Demonstrated competency in ${pSkill.name}`,
74 |           confidence: foundSkill.confidence || 0.9,
75 |           category: pSkill.category || 'Preferred Advantage',
76 |         });
77 |       } else {
78 |         missingPreferredSkills.push(pSkill.name);
79 |       }
80 |     }
81 |
82 |     // Skill alignment calculation:
83 |     // If mandatory skills exist, achieving 100% of mandatory skills gives a solid 85 base
   >  score
84 |     // Preferred skills provide the remaining 15 bonus points up to 100
85 |     let mandatoryRatio = mandatoryList.length > 0 ? (matchedRequiredSkills.length / mandat
   > oryList.length) : 1;
86 |     let preferredRatio = preferredList.length > 0 ? (matchedPreferredSkills.length / prefe
   > rredList.length) : 1;
87 |
88 |     let skillAlignmentScore = Math.round((mandatoryRatio * 85) + (preferredRatio * 15));
89 |     skillAlignmentScore = Math.max(10, Math.min(100, skillAlignmentScore));
90 |
91 |     // 3. Semantic Similarity via AI Provider
92 |     const jobFullText = `${job.title} ${job.department} ${job.rawDescription || ''} ${(job
   > .responsibilities || []).join(' ')} ${(mandatoryList.map((m) => m.name)).join(' ')}`;
93 |     const semanticSimilarityScore = await aiProvider.calculateSimilarity(resumeText, jobFu
   > llText);
94 |
95 |     // 4. Experience Alignment
96 |     const candidateYears = candidate.totalExperienceYears || 0;
97 |     const requiredYears = job.minYearsExperience || 1;
98 |
99 |     let experienceScore = 100;
100 |     if (candidateYears >= requiredYears) {
101 |       experienceScore = 100;
102 |     } else if (candidateYears >= requiredYears * 0.75) {
103 |       experienceScore = 80;
104 |     } else if (candidateYears >= requiredYears * 0.5) {
105 |       experienceScore = 60;
106 |     } else {
107 |       experienceScore = Math.max(30, Math.round((candidateYears / requiredYears) * 60));
108 |     }
109 |
110 |     const experienceSummary = candidateYears >= requiredYears
111 |       ? `Meets experience requirement (${candidateYears} yrs vs ${requiredYears} yrs minim
   > um)`
112 |       : `Experience below target (${candidateYears} yrs vs ${requiredYears} yrs required)`
   > ;
113 |
114 |     // 5. Qualification Alignment
115 |     let highestCandidateDegree = "Bachelor's Degree";
116 |     if (candidate.educationTimeline && candidate.educationTimeline.length > 0) {
117 |       highestCandidateDegree = candidate.educationTimeline[0].degree || "Bachelor's Degree
   > ";
118 |     }
119 |
120 |     const requiredDegreeStr = (job.qualifications && job.qualifications.length > 0) ? job.
   > qualifications[0] : "Bachelor's Degree";
121 |     const candRank = getDegreeRank(highestCandidateDegree);
122 |     const reqRank = getDegreeRank(requiredDegreeStr);
123 |
124 |     let qualificationScore = 80;
125 |     let matchStatus = 'Meets';
```

```
126 |
127 |    if (candRank > reqRank) {
128 |      qualificationScore = 100;
129 |      matchStatus = 'Exceeds';
130 |    } else if (candRank === reqRank) {
131 |      qualificationScore = 95;
132 |      matchStatus = 'Meets';
133 |    } else if (candRank === reqRank - 1) {
134 |      qualificationScore = 75;
135 |      matchStatus = 'Equivalent';
136 |    } else {
137 |      qualificationScore = 55;
138 |      matchStatus = 'Below';
139 |    }
140 |
141 |    // 6. Overall Weighted Score Calculation
142 |    const overallScore = Math.round(
143 |      (skillAlignmentScore * normSkillsWeight) +
144 |      (semanticSimilarityScore * normSemanticWeight) +
145 |      (experienceScore * normExpWeight) +
146 |      (qualificationScore * normQualWeight)
147 |    );
148 |
149 |    // 7. Extract Supporting Resume Excerpts
150 |    const supportingExcerpts = [];
151 |    const sentences = resumeText.split(/(?<=[.?!])\s+|\n+/).filter((s) => s.trim().length
    >  > 25);
152 |
153 |    // Excerpt for top skills
154 |    matchedRequiredSkills.slice(0, 3).forEach((item) => {
155 |      supportingExcerpts.push({
156 |        topic: `Required Skill: ${item.skill}`,
157 |        quote: item.evidence,
158 |        sourceSection: 'Skills & Experience',
159 |      });
160 |    });
161 |
162 |    if (supportingExcerpts.length < 3 && sentences.length > 0) {
163 |      supportingExcerpts.push({
164 |        topic: 'Core Accomplishment',
165 |        quote: sentences[0].substring(0, 140) + '...',
166 |        sourceSection: 'Profile Summary',
167 |      });
168 |    }
169 |
170 |    // 8. Data Quality and Responsible Warnings
171 |    const dataQualityWarnings = [];
172 |    if (missingRequiredSkills.length > 0) {
173 |      dataQualityWarnings.push(`Missing ${missingRequiredSkills.length} mandatory skill(s)
    >  : ${missingRequiredSkills.join(', ')}`);
174 |    }
175 |    if (candidateYears < requiredYears) {
176 |      dataQualityWarnings.push(`Years of experience (${candidateYears} yrs) is under the r
    >  ole's requested ${requiredYears} years.`);
177 |    }
178 |    if (semanticSimilarityScore > 85 && missingRequiredSkills.length > 2) {
179 |      dataQualityWarnings.push('High semantic match despite missing core mandatory skills;
    >   verify practical hands-on experience.');
180 |    }
181 |
182 |    return {
183 |      overallScore: Math.min(100, Math.max(15, overallScore)),
184 |      componentScores: {
185 |        skillAlignment: skillAlignmentScore,
186 |        semanticSimilarity: semanticSimilarityScore,
187 |        experienceScore: experienceScore,
188 |        qualificationScore: qualificationScore,
189 |      },
190 |      weightsUsed: weights,
191 |      matchedRequiredSkills,
192 |      matchedPreferredSkills,
193 |      missingRequiredSkills,
```

```
194 |      missingPreferredSkills,
195 |      experienceEvidence: {
196 |        candidateYears,
197 |        requiredYears,
198 |        evidenceExcerpt: `${candidateYears} years of cumulative relevant industry experien
    >   ce.`,
199 |        summary: experienceSummary,
200 |      },
201 |      qualificationEvidence: {
202 |        candidateDegree: highestCandidateDegree,
203 |        requiredDegree: requiredDegreeStr,
204 |        matchStatus,
205 |        evidenceExcerpt: `Verified credential: ${highestCandidateDegree}`,
206 |      },
207 |      supportingExcerpts,
208 |      dataQualityWarnings,
209 |    };
210 |  }
211 |
212 |  module.exports = {
213 |    calculateCandidateMatch,
214 |    getDegreeRank,
215 |  };
216 |
```